國立雲林科技大學　介面設計課程

期中專題報告

臺鐵乘車資訊服務重新設計

從 Bad UI 分析到 Django × Neon 全端實作

Bad UI Case：國營臺灣鐵路股份有限公司官方網站

https://www.railway.gov.tw/

項目	內容	項目	內容
組別	第 13 組	組員	曾宥綸、吳嘉恩、陳柏宏、王慈文
成果網站	https://railway-redesign.onrender.com/	管理後台	https://railway-redesign.onrender.com/admin/
專案倉庫	https://github.com/Yuntech-work/hw1-railway-redesign	繳交日期	2026 年 10 月 27 日

正式站截圖為 2026/10/27 實際操作畫面；Bad UI 截圖為 2026/10/09；AI 對話紀錄節錄自本專案 OpenCode session 匯出檔。

摘要

本專題以「國營臺灣鐵路股份有限公司官方網站」為 Bad UI 案例，先以實際操作、截圖、Lighthouse 檢測與三類 Persona 情境走查，歸納出 8 項介面問題；再將問題轉化為 8 條使用者需求，完成 12 張重新設計分鏡，並進一步實作一套可操作的全端系統。

系統以 Django 5.2 作為應用層，採 server-rendered templates 延續原型的 Design System；會員、車站、班次、車廂、票種、票價、訂單、乘車人、公告與服務狀態等 15 個模型儲存於 Neon PostgreSQL 18，後台使用 Django Admin 管理。前台提供任務導向首頁、多條件查詢、班次比較、票價說明、三步驟訂票、電子票 QR code、會員中心與服務異常替代方案；訪客可查詢，登入後才能訂票，訂票交易以資料庫鎖定避免超賣。

系統部署於 Render，前台、後台與健康檢查均已上線；Django 13 個單元測試全數通過，公開站端對端 smoke test 20 項全數 PASS。報告亦收錄 11 組實際 AI Prompt 紀錄，說明每組的使用目的、AI 回覆與對應實作成果，並以 axe-core 4.12.1 進行無障礙檢測，誠實列出仍待改善的項目。

關鍵詞：Bad UI、使用者經驗、Django、Neon PostgreSQL、Django Admin、Render、無障礙、AI 輔助開發

目錄

1	**需求說明**	專題名稱、組員、網站主題、目標與驗收對照
2	**Bad UI 分析結果**	原始網站問題、使用者痛點、Lighthouse 與無障礙檢測
3	**改善目的與網站規格**	改善目標、系統架構、網站地圖、Design System、資料模型
4	**前端實作展示**	首頁、查詢、結果比較、會員登入前後、訂票流程、服務狀態
5	**後台與資料庫**	Django Admin、Neon PostgreSQL、資料管理與安全性
6	**AI Prompt 使用紀錄（11 組）**	實際對話節錄、使用目的、AI 回覆與對應成果
7	**部署與測試結果**	Render 部署、建置、13 個測試、20 項 smoke test、axe 檢測
8	**結論與心得**	改善成果、遭遇問題、組員心得與未來展望
附錄 A	**路由、帳號與環境變數**	完整路由、測試帳號、部署環境
附錄 B	**AI 工具使用與人工覆核**	工具名稱、使用範圍、覆核方式
附錄 C	**參考資料**	官方文件、標準與檢測工具

閱讀說明

一、本報告的「圖 x-x」為實際系統截圖或檢測結果；「表 x-x」為整理後的資料與規格。

二、第 6 章的 AI 對話卡片，為統一版面而重新排版，文字逐字節錄自 OpenCode 實際 session；每張卡片均標示 Session ID，可用 opencode session export 重新匯出比對。

三、Render 免費方案閒置約 15 分鐘會休眠，首次開啟網站可能需 30–60 秒喚醒，這是平台限制而非系統錯誤。

1. 需求說明

1.1　專題名稱與網站主題

本專題名為「臺鐵乘車資訊服務重新設計：從 Bad UI 分析到 Django × Neon 全端實作」。以臺鐵官網為 Bad UI 案例，重新設計乘車資訊查詢與線上訂票流程，並將設計落實在一個可登入、可查詢、可訂票、可由後台管理的完整網站。

項目	內容
專題名稱	臺鐵乘車資訊服務重新設計：從 Bad UI 分析到 Django × Neon 全端實作
網站主題	公共運輸乘車資訊與線上訂票服務（以臺鐵官網為案例的重新設計）
原始案例	國營臺灣鐵路股份有限公司官方網站（https://www.railway.gov.tw/）
成果網站	https://railway-redesign.onrender.com/（前台）｜https://railway-redesign.onrender.com/admin/（後台）
專案倉庫	https://github.com/Yuntech-work/hw1-railway-redesign
核心任務	查班次 → 比較車次 → 看票價與優惠 → 選車廂 / 票種 → 填乘車人 → 完成訂票 → 取得電子票 → 查我的訂票
目標使用者	都會通勤族、熟齡旅客、首次線上訂票者 / 外國旅客
技術棧	Django 5.2、Neon PostgreSQL 18、Django Admin、Render、Gunicorn、WhiteNoise、HTML / CSS / Vanilla JS

1.2　組員與分工

本專題由第 13 組四位組員共同完成，分工依「分析與設計、流程與會員、後端與資料庫、部署與報告」四個面向劃分，貢獻比例各 25%。

表 1-1　組員分工與貢獻比例

組員	學號	負責面向	主要工作	比例
曾宥綸	B11317017	**分析與介面設計**	Bad UI 案例選擇、7 張原始截圖、P1–P8 問題分析、Persona 與資訊架構；前台首頁與查詢表單的設計對照。	25%
吳嘉恩	B11317032	**流程與會員功能**	User Story、Scenario、三組 User Flow；查詢結果、比較、票價、會員登入前後與我的訂票功能驗收。	25%

組員	學號	負責面向	主要工作	比例
陳柏宏	B11317040	**後端與資料庫**	Django models / views / forms、三步驟訂票交易、Django Admin 自訂、Neon 連線與 seed_demo 種子資料。	25%
王慈文	B11317042	**部署、測試與報告**	Render 部署、環境變數、smoke test、axe 無障礙檢測、AI Prompt 紀錄整理、Word / PDF 版面編排。	25%

全組貢獻比例合計：100%。

1.3 網站範圍與功能定位

- **前台**：首頁（預設 / 大字 / 新手導引）、查詢表單、查詢結果、班次比較、票價與優惠、車廂選擇、票種與人數、乘車人與付款、訂票完成、電子票 QR code、服務狀態、會員登入 / 註冊、我的訂票、會員資料。
- **後台**：Django Admin 管理會員與權限、車站、車種、列車、班次、車廂、票種、票價規則、訂單、乘車人、常用乘車人、公告與服務狀態；訂單可標記付款、取消並釋出座位。
- **資料庫**：Neon PostgreSQL 18 的 production branch，前台查詢與後台管理共用同一套 Django ORM 與資料來源。
- **不在範圍內**：真實金流、真實臺鐵閘門驗票、簡訊 / Email 通知、單一座位選位圖、多語系完整翻譯。付款為課程 Demo 的模擬付款，不儲存信用卡資料。

1.4 專題目標與驗收對照

下表把期中專題的基本要求逐項對照到本專題的實作與可驗收證據，確保報告不是只有畫面，而是有實際功能與測試結果。

表 1-2　期中專題要求與實作對照

作業要求	本專題實作	驗收證據
前台至少 3–5 個頁面	12 條以上前台路由：首頁、查詢、結果、比較、票價、訂票 3 步、完成、服務狀態、登入、註冊、我的訂票、會員資料	第 4 章截圖、附錄 A 路由表
會員登入前後差異	訪客可查詢 / 比較 / 看票價；訂票需登入；會員有我的訂票、會員資料、常用乘車人自動帶入	圖 4-5～4-7、表 3-5
多元資料查詢、篩選、搜尋	車站 autocomplete、日期、時段、車種、優惠票、有位、可帶寵物、票價排序、分頁、2–3 班比較	圖 4-2～4-4、表 3-6

作業要求	本專題實作	驗收證據
Django 管理後台	13 個模型註冊、MemberAdmin / BookingAdmin / TripAdmin 自訂、訂單取消退座 action	圖 5-1～5-4
Neon PostgreSQL	Neon production branch、24 張 public tables、psycopg2、pooled connection	表 5-2、表 5-3
雲端部署前後台連結	Render Web Service，前台、/admin/、/healthz/ 均上線	表 7-1、表 7-2
至少 10 組 AI Prompt	11 組實際 Prompt，含對話節錄、目的、AI 回覆與成果	第 6 章
測試會員與管理員帳密	user / user123、admin / admin123	表 1-3

1.5　測試帳號與線上入口

表 1-3　測試帳號與線上入口

項目	帳號 / 網址	說明
會員	user / user123	可訂票、查看電子票、我的訂票、會員資料
管理員	admin / admin123	可進入 /admin/ 管理所有資料
前台	https://railway-redesign.onrender.com/	Render 免費方案，首次開啟可能需 30–60 秒
後台	https://railway-redesign.onrender.com/admin/	Django Admin 登入頁
健康檢查	https://railway-redesign.onrender.com/healthz/	回傳 {"status":"ok","database":"ok"} 代表應用與資料庫均正常
專案倉庫	https://github.com/Yuntech-work/hw1-railway-redesign	GitHub main 分支

展示提醒

Render 免費方案閒置約 15 分鐘會休眠，第一次開啟可能需要 30–60 秒；建議展示前先開啟 /healthz/ 喚醒服務。

所有表單與交易均為課程 Demo，付款流程為模擬付款，不會傳送或儲存真實信用卡資料。

2. Bad UI 分析結果

2.1　案例選擇與調查方法

本專題選擇臺鐵官網作為案例，原因包括：使用規模大、問題可實際觀察、決策點密集適合設計 User Flow，以及使用者樣貌多元（通勤族、熟齡旅客、首次訂票者）。這些問題不是抽象的主觀偏好，而是可以在實際操作與檢測中重現。

本專題未進行大規模問卷調查，而是採「啟發式評估 + 客觀檢測 + Persona 情境走查」三角驗證，避免把個人喜好當成使用者需求：

- **實際操作與截圖：**實際開啟臺鐵官網，走完首頁、時刻查詢、送出查詢、結果列表等主要流程，擷取 7 張原始介面截圖。
- **啟發式評估：**以資訊層級、導覽、版面配置、一致性、操作回饋、可讀性六個面向逐項檢視，並標註可佐證的截圖編號。
- **Lighthouse 檢測：**於查詢結果頁執行 Lighthouse，取得 Accessibility 0.89、Best Practices 0.83、SEO 0.80，作為客觀佐證。
- **Persona 情境走查：**建立都會通勤族、熟齡旅客、首次線上訂票者三位 Persona，逐項確認問題是否真的會阻斷他們的任務。
- **無障礙樹檢查：**以瀏覽器無障礙樹檢視查詢表單，發現所有控制項在樹中呈現為無名稱的 generic 節點。

2.2　原始介面截圖

以下 4 張截圖涵蓋首頁、導覽、驗證錯誤與查詢結果，是後續 P1–P8 問題分析的主要證據。



圖 2-1　官網首頁：Cookie 同意橫幅壓在畫面最下方，主視覺被大型輪播廣告佔滿，主要任務入口不明顯。



圖 2-2　官網首頁：右側固定浮動選單文字被視窗邊界裁切，「臺鐵購物工場」換行溢出、部分文字遭截斷。



圖 2-3　送出查詢後的驗證錯誤：只顯示「出發站格式錯誤 / 抵達站格式錯誤」，未說明真正原因（必須從自動完成清單選取）。



圖 2-4　查詢結果表格：欄位多、字級小，右側固定浮動選單直接遮住最右欄「訂票」，使用者看得到車次卻點不到訂票。

2.3　八項問題分析

下表將 8 項問題整理為「問題類別、現象與截圖佐證、使用者影響、對應改善」，每項都能對應到第 3 章的需求與第 4 章的實作畫面。

表 2-1　Bad UI 八項問題分析

編號	類別	問題現象與截圖佐證	對使用者的影響	對應改善
P1	**資訊層級**	首頁視覺焦點是大型輪播廣告，最高頻的「查詢班次 / 訂票」只是頁面下半部的小表單，且夾在新聞專區之下。	使用者進入網站前 5 秒無法判斷主要任務入口，放棄率上升。	首頁改以「我要做的事」三張任務卡作為唯一主要入口，輪播降級為次要資訊。
P2	**導覽**	頂部 7 個等權重主選單，右側又有一組固定浮動選單重複導覽，兩組沒有明確分工。	選單過多造成選擇負擔，重複導覽增加維護成本且容易不一致。	頂部導覽精簡為 5 項並標示目前位置，移除右側浮動選單。
P3	**版面配置**	右側浮動選單固定寬度卻未避開內容區，在 1280px 以上直接覆蓋結果表格最右側的「訂票」欄。	主要任務按鈕被遮住，屬於功能性阻斷，使用者無法完成訂票。	移除浮動選單，結果改為卡片式列表，訂票按鈕固定於每張卡片右側。
P4	**一致性**	頁面同時出現深藍、淺藍、亮綠、橘紅、深紅五種以上強調色；錯誤、公告、警語與一般內文混用紅 / 黑字。	色彩無法承載語意，使用者難以快速分辨「警示」與「強調」。	全站收斂為 1 種品牌藍、1 種行動橘與 3 種狀態色，語意固定。
P5	**操作回饋**	車站欄位可自由輸入，但系統要求必須從自動完成清單選取；送出後只回「格式錯誤」，維護頁也只有「回首頁」一個按鈕。	使用者反覆嘗試仍無法完成任務，信任感下降，轉而放棄線上服務。	欄位即時驗證並寫明正確作法；異常頁提供恢復時間、三種替代通路與重試按鈕。
P6	**可讀性**	查詢結果表格字級偏小、欄位密集（10 欄），次要文字使用低對比灰色；Lighthouse 回報連結僅依賴顏色區辨。	熟齡與低視力使用者閱讀困難，可能誤判哪些文字可點擊。	內文對比提升、次要文字加大、連結一律加下劃線，並提供全域大字模式。
P7	**無障礙**	查詢表單的控制項在無障礙樹中全為無名稱的 generic 節點；文件缺少 main 地標，清單結構錯誤。	螢幕閱讀器使用者無法得知每個欄位要填什麼，實質上無法使用查詢。	所有欄位以 label 正確關聯、補上必填標記與 aria-describedby，新增 main 地標與跳至主要內容。
P8	**查詢脈絡**	捲動到結果表格後，原始查詢條件（出發站、抵達站、日期、時段）已離開畫面，也不顯示查詢時間與結果筆數。	使用者需反覆捲動確認條件，增加認知負荷與誤訂風險。	結果頁頂部設固定查詢摘要列，顯示路線、日期、時段與結果筆數，並提供「修改條件」。

2.4　使用者痛點歸納

同樣的介面問題，對不同使用者的傷害程度不同。下表把三類 Target User 的主要痛點對應到問題編號，作為後續 Persona 與 User Story 的依據。

表 2-2　三類使用者痛點與對應問題

使用者	主要任務	核心痛點	對應問題
都會通勤族	在 30 秒內找到最快抵達且仍有優惠票的車次並完成訂票	首頁找不到訂票入口；結果表格的訂票欄被浮動選單遮住；捲動後忘記查的是哪一天	P1、P3、P8
熟齡旅客	看懂票價與優惠資格、確認應備證件、選擇合適車廂	字太小、灰字看不清；優惠規則分散在不同頁；不知道優惠票必須搭配全票	P4、P6
首次訂票者 / 外國旅客	在不懂規則的情況下完成第一次訂票，並確認付款安全	不知道車站必須從清單選取；事前不知道要填身分證；付款前沒有防詐提示	P5、P7

2.5　客觀檢測數據

為避免只憑主觀印象判斷，於查詢結果頁執行 Lighthouse，並以瀏覽器無障礙樹檢查表單。結果如下：

表 2-3　Lighthouse 檢測結果（原始網站查詢結果頁）

檢測項目	分數	說明
Accessibility（無障礙）	0.89	未達滿分，存在可修正的無障礙缺陷
Best Practices（最佳實務）	0.83	包含圖片比例、連結可抓取等問題
SEO（搜尋最佳化）	0.80	連結未使用有效 href，影響可爬性

Lighthouse 回報的具體失敗項目：

- Links rely on color to be distinguishable（連結僅靠顏色區辨）→ 對應 P6 可讀性
- Document does not have a main landmark（缺少 main 地標）→ 對應 P7 無障礙
- Lists do not contain only <li> / List items not contained within （清單結構錯誤）→ 對應 P7
- Links are not crawlable（連結不可抓取）→ 對應 P2 導覽
- Displays images with incorrect aspect ratio（圖片比例失真）→ 對應 P3 版面配置

2.6　小結

原始網站最嚴重的問題不是「不好看」，而是「主要任務被阻斷」：訂票按鈕被浮動選單遮住（P3）、錯誤訊息無法指引修正（P5）、表單在無障礙樹中沒有名稱（P7）。這三項都直接讓使用者無法完成任務，因此重新設計時優先處理「任務入口、操作回饋、無障礙名稱」，再處理視覺一致性與可讀性。

3. 改善目的與網站規格

3.1 改善目標

改善目標不是「把網站變漂亮」，而是把第 2 章的問題轉成可驗收的需求。下表將 P1–P8 對應到 N1–N8，並標示本專題在那一個頁面解決它。

表 3-1 改善目標與問題對照

編號	改善目標	對應問題	實作頁面 / 元件
N1	能在短時間內比較多個候選車次的時間與票價	P1、P3	首頁任務卡、查詢結果卡片、班次比較頁
N2	主要任務入口必須明確且不被遮擋	P1、P2、P3	首頁「我要做的事」、固定導覽、卡片式結果
N3	隨時知道目前查詢條件與結果範圍	P8	結果頁查詢摘要列、修改條件、分頁
N4	足夠的字級與色彩對比，能清楚閱讀	P4、P6	Design System、大字模式、高對比色票
N5	在使用優惠票前，先確認資格與應備證件	P4、P6	票價與優惠頁、票種卡片、資格說明
N6	在每一步都知道進度、必填欄位與規則限制	P5、P7	三步驟指示器、即時驗證、label 關聯
N7	在輸入敏感資料與完成交易後獲得明確回饋	P5	乘車人頁防詐提示、完成頁成功回饋與 QR code
N8	服務異常時仍能完成任務的替代方案	P5	服務狀態頁、維護公告、替代通路

3.2　系統架構

系統採三層架構：瀏覽器負責呈現與互動；Django 負責路由、樣板、會員驗證、表單驗證、訂票交易與 Admin；Neon PostgreSQL 負責持久化。前台與後台共用同一套 ORM 與資料庫，避免「展示用資料」與「管理用資料」不一致。

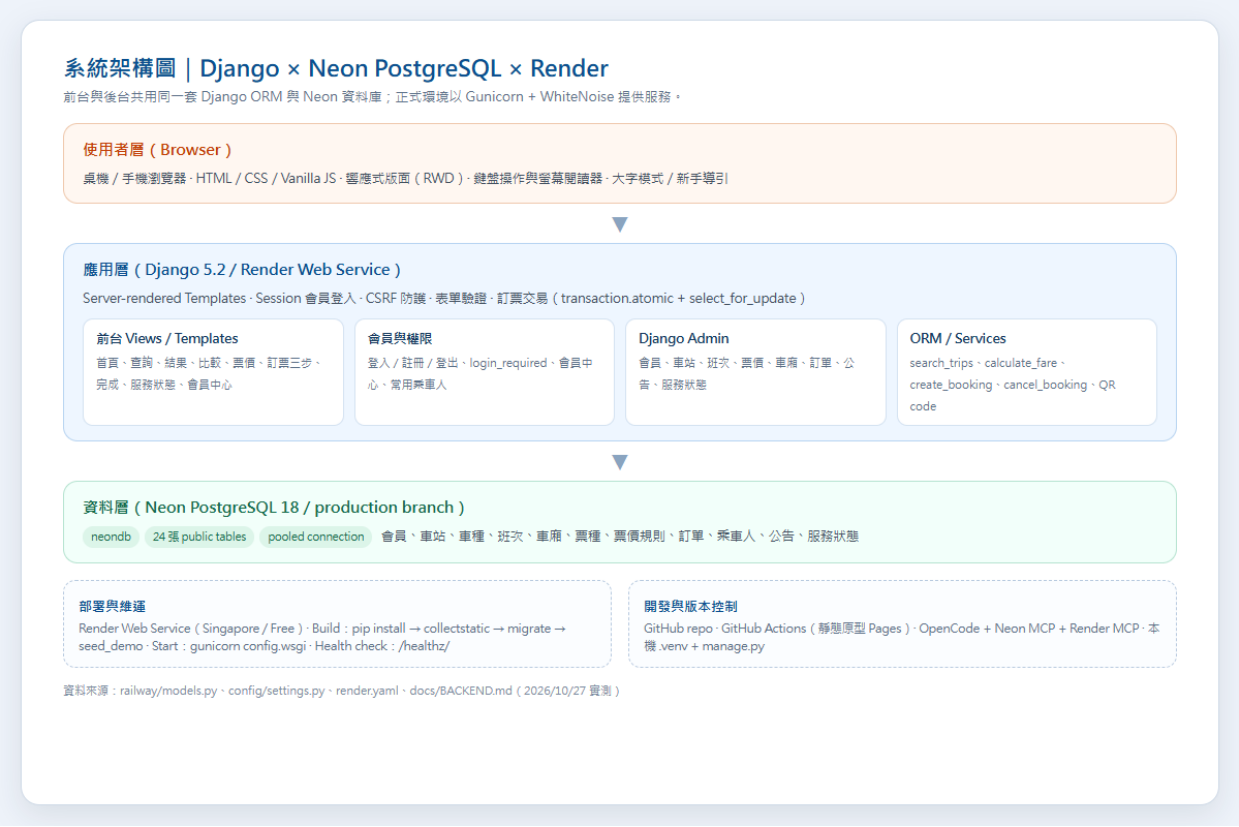


圖 3-1　系統架構圖：Browser → Django（Views / Templates / Auth / Admin / ORM）→ Neon PostgreSQL，部署於 Render。

正式環境以 Gunicorn 啟動 Django，WhiteNoise 提供靜態檔案，Render 以 /healthz/ 作為健康檢查；資料庫連線使用 Neon 的 pooled connection，並以 psycopg2 作為驅動。所有部署參數都由環境變數注入，不把資料庫密碼或 SECRET_KEY 寫進版本控制。

3.3　網站地圖與路由

本專題實作 12 條以上前台路由與完整的後台入口，涵蓋「查詢 → 比較 → 訂票 → 取票 → 查訂單」的主要任務，以及服務異常與會員資料等支援頁面。

表 3-2　主要路由與功能

路由	功能
/	首頁、任務卡、公告、快速查詢；?mode=large 大字、?mode=novice 新手導引
/search/	訂票查詢表單、車站 autocomplete、交換站、日期與時段

路由	功能
`/search/results/`	班次列表、車種 / 優惠 / 座位 / 寵物篩選、票價排序、分頁、勾選比較
`/compare/`	2–3 班次並排比較出發 / 抵達時間、行駛時間、票價與車廂
`/fares/`	票價規則、票種資格、電子票折扣
`/booking/<trip_id>/carriage/`	車廂選擇（自由座 / 指定座 / Coleman / 商務）
`/booking/<trip_id>/ticket/`	票種、乘客人數、電子票、條款確認
`/booking/<trip_id>/passenger/`	乘車人 formset、付款方式、建立訂單
`/booking/complete/<order_no>/`	訂單明細、成功回饋、電子票 QR code
`/status/`	服務狀態、維護公告、替代通路；?demo=maintenance 預覽維護
`/accounts/login/`、`/signup/`、`/logout/`	會員登入、註冊、登出
`/my/bookings/`、`/my/profile/`	我的訂票、取消訂票、會員資料、常用乘車人
`/admin/`	Django 管理後台
`/api/stations/?q=`、`/healthz/`	車站 autocomplete JSON、健康檢查

3.4 頁面設計與 Design System

為了讓 12 張分鏡與正式系統維持一致，先定義單一 Design System，再套用到 Django templates。正式系統沿用原型 styles.css 的 design tokens，僅補上 Django 表單錯誤、會員中心、分頁與 QR code 樣式。

表 3-3　Design System 規格

項目	規格
色彩	品牌藍 #0B4F8F（導覽與標題）、行動橘 #C43E06（主要 CTA）、成功 #256B3C、注意 #7A4B00、錯誤 #B3261E、內文 #16202B、次要 #43505F、背景 #F3F6FA
字體	Noto Sans TC / PingFang TC / Microsoft JhengHei；字級級距 13 / 15 / 16 / 18 / 22 / 30 / 40 px，內文基準 16px
對比	主要文字對比 15.8:1、次要文字 8.3:1、主要按鈕白字 5.6:1，符合 WCAG 2.1 AA
按鈕	主要（橘色實心）、次要（藍色實心）、外框、文字四種；最小點擊高度 48px、圓角 8px
導覽	桌面 5 項主導覽 + 工具列（大字 / 登入）；手機改為單列可水平捲動，不再讓每個標籤斷行
版面	內容最大寬度 1180px 置中、12 欄網格；間距 4 / 8 / 12 / 16 / 24 / 32 / 48 px；卡片圓角 14px
無障礙	label for 關聯、必填標記、aria-describedby、skip link、focus 3px、main 地標、SVG 圖示 aria-hidden
大字模式	全域放大 18%，版面與元件不變，只放大字級與間距；選擇結果會被記住

3.5　資料模型

資料模型以「會員、運輸資源、票務、交易、內容」五個群組劃分，共 15 個模型。會員繼承 AbstractUser，訂單與乘車人分離，常用乘車人可重複使用；班次與車廂分離，才能分別管理票價與剩餘座位。

表 3-4　資料模型一覽

Model	說明
Member	自訂會員（繼承 AbstractUser）：手機、生日、身分證、字級偏好、會員等級
Station	車站代碼、名稱、英文名、城市、是否啟用
TrainType	自強號、莒光號、區間車、普悠瑪、太魯閣
Train	車次、車種、經由、行駛時間、可帶寵物、餐車
Trip	某日某區間班次：起訖時間、票價、座位數、狀態、是否建議
CarriageType	標準自由座、標準指定座、Coleman、商務車廂
TripCarriage	班次 × 車廂的票價與剩餘座位
TicketType	全票、優惠票、敬老、學生、兒童、愛心票與資格
FareRule	區間票價規則、電子票折扣
Booking	訂單、會員、班次、車廂、票種、金額、付款、狀態、QR token
Passenger	訂單乘車人姓名、證件、生日、手機、Email
SavedPassenger	會員常用乘車人，下次訂票自動帶入
Announcement	首頁與服務頁公告、維護通知
ServiceStatus	訂票 / 取票 / 網站系統狀態與預計恢復時間
FavoriteRoute	會員常用路線（預留）

3.6　會員與權限設計

會員機制採 Django 內建 session 與 login_required，依「訪客 / 會員 / 管理員」三種角色區分功能。訪客仍可查詢與比較，降低使用門檻；涉及個資與交易的動作才要求登入，避免一開始就阻擋使用者。

表 3-5　三種角色的功能權限

功能	訪客	會員	管理員
查詢班次、票價、比較	可	可	可

功能	訪客	會員	管理員
訂票、填乘車人、付款	導向登入	可	可
電子票 QR code、我的訂票	不可	可	可
取消訂票、退還座位	不可	可（自己的訂單）	可（所有訂單）
管理會員、班次、票價、公告	不可	不可	可（/admin/）

3.7　查詢與訂票流程規格

查詢支援 9 個條件與 3 種排序；訂票分成「車廂 → 票種與人數 → 乘車人與付款 → 完成」四個畫面，但以三步驟指示器呈現，讓使用者知道自己在哪裡。建立訂單時以資料庫交易鎖定班次與車廂，避免同時訂票造成超賣。

表 3-6　查詢與訂票流程規格

階段	使用者操作	系統行為與驗證
查詢	選出發站、抵達站、日期、時段、票種	車站 autocomplete、交換站、日期範圍 15 天、時段檢核
篩選	車種、優惠票、有位、可帶寵物、票價排序、分頁	伺服器端查詢與 filter chips；結果頁固定顯示查詢摘要
比較	勾選 2–3 班，進入比較頁	並排顯示時間、行駛時間、票價、車廂與電子票適用
選車廂	選自由座 / 指定座 / Coleman / 商務	檢查剩餘座位；售完車廂不可選，提供替代建議
選票種	全票 / 優惠票 / 敬老 / 學生 / 兒童 / 愛心	顯示資格與應備證件；優惠票需搭配全票；即時計算金額
乘車人	填姓名、身分證、生日、手機、Email、付款方式	伺服器端格式驗證、必填檢核、條款勾選、防詐提示
建立訂單	按下確認	transaction.atomic + select_for_update 鎖定班次與車廂、扣減座位、產生訂單編號與 QR token
完成	查看訂單明細與 QR code	成功回饋、取票說明、可下載 / 顯示電子票；我的訂票可取消並退座

4. 前端實作展示

本章以正式站（Render）的實際畫面展示 5 個核心頁面與會員登入前後差異。桌面版截圖為 2026/10/27 以 1440×900 視窗實際操作擷取，手機版為 iPhone 15 尺寸，皆非設計稿模擬。

4.1　首頁：任務導向入口

首頁把使用者最常做的事放在最前面：三張「訂車票 / 查詢班次 / 票價與優惠」任務卡，取代原本被輪播廣告佔滿的首屏。上方狀態列顯示系統正常與最後更新時間；公告區只保留與旅程相關的電子票與維護訊息。

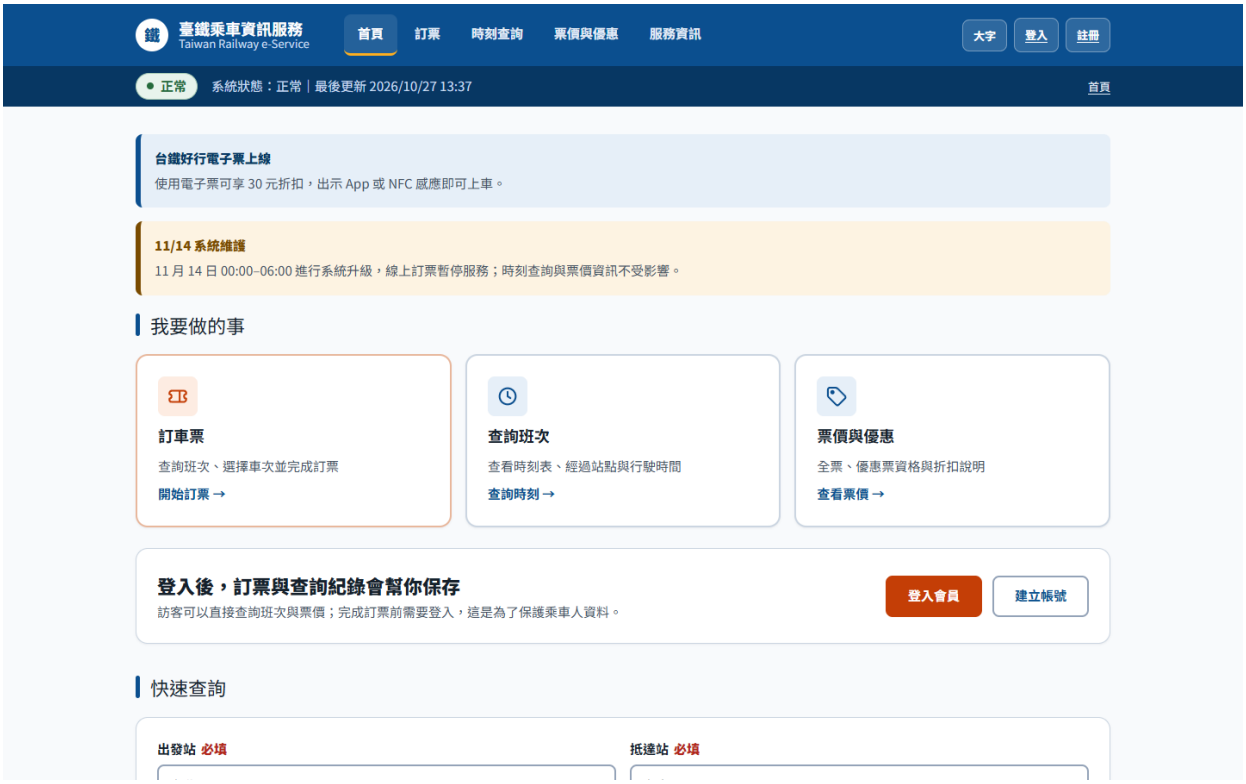


圖 4-1　首頁（訪客）：三張任務卡是主要入口，右上角提供大字、登入與註冊；下方仍有快速查詢表單。

訪客可以看到「登入後，訂票與查詢紀錄會幫你保存」的提示，但不需要登入就能查班次與票價。這個設計降低第一次使用者的門檻，也符合「先給價值、再要求註冊」的原則。

4.2　查詢表單：多元條件與即時提示

查詢表單將出發站、抵達站、日期、時段、票種集中在同一張卡片，車站欄位提供 autocomplete 與「輸入後請從建議清單中選擇」的說明，避免原站「自行輸入後只回格式錯誤」的問題。交換站按鈕讓使用者不用重新輸入兩站。

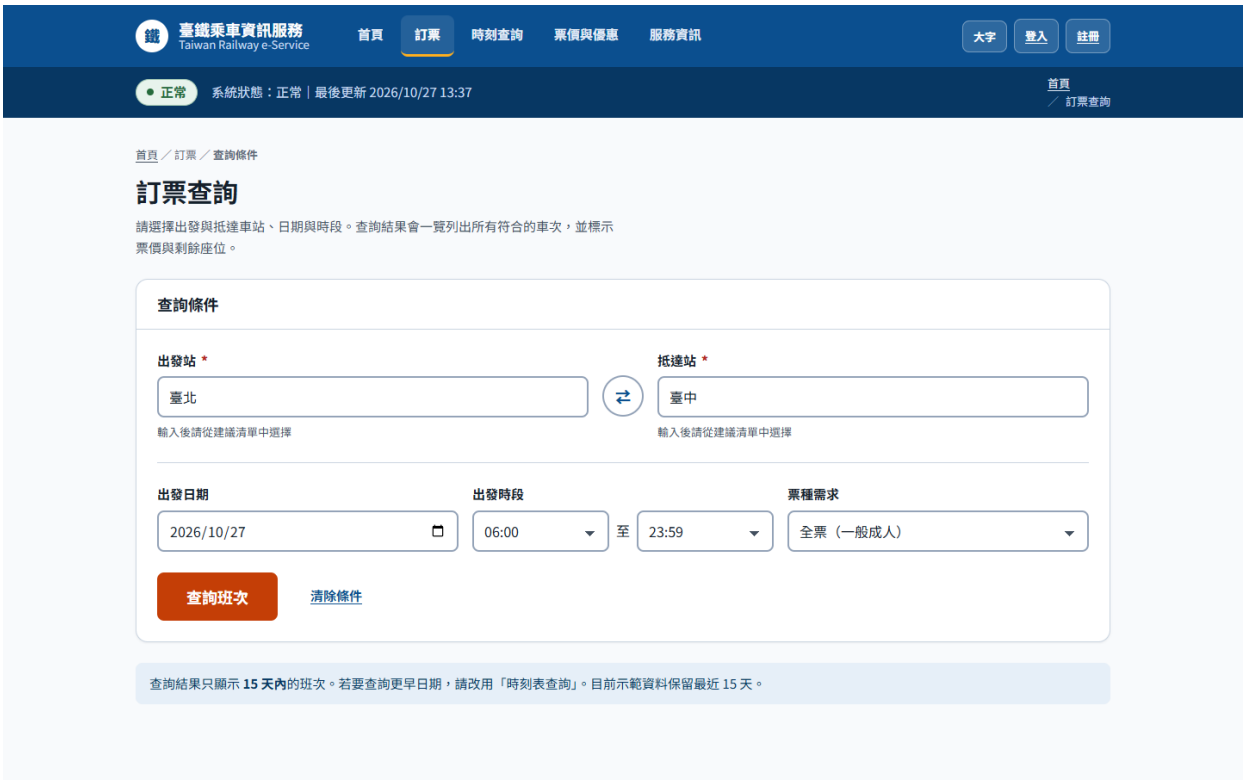


圖 4-2　訂票查詢表單：車站 autocomplete、交換站、日期與時段、票種需求集中在同一張卡片。

4.3　查詢結果與班次比較

結果頁頂部固定顯示「臺北 → 台中、2026/10/28、共 24 班」的查詢摘要，往下捲動也不會失去脈絡。每張班次卡片右側直接放「訂票」按鈕，不再被浮動選單遮擋；快速篩選以 chips 呈現，可切換車種、優惠票、有位與可帶寵物。

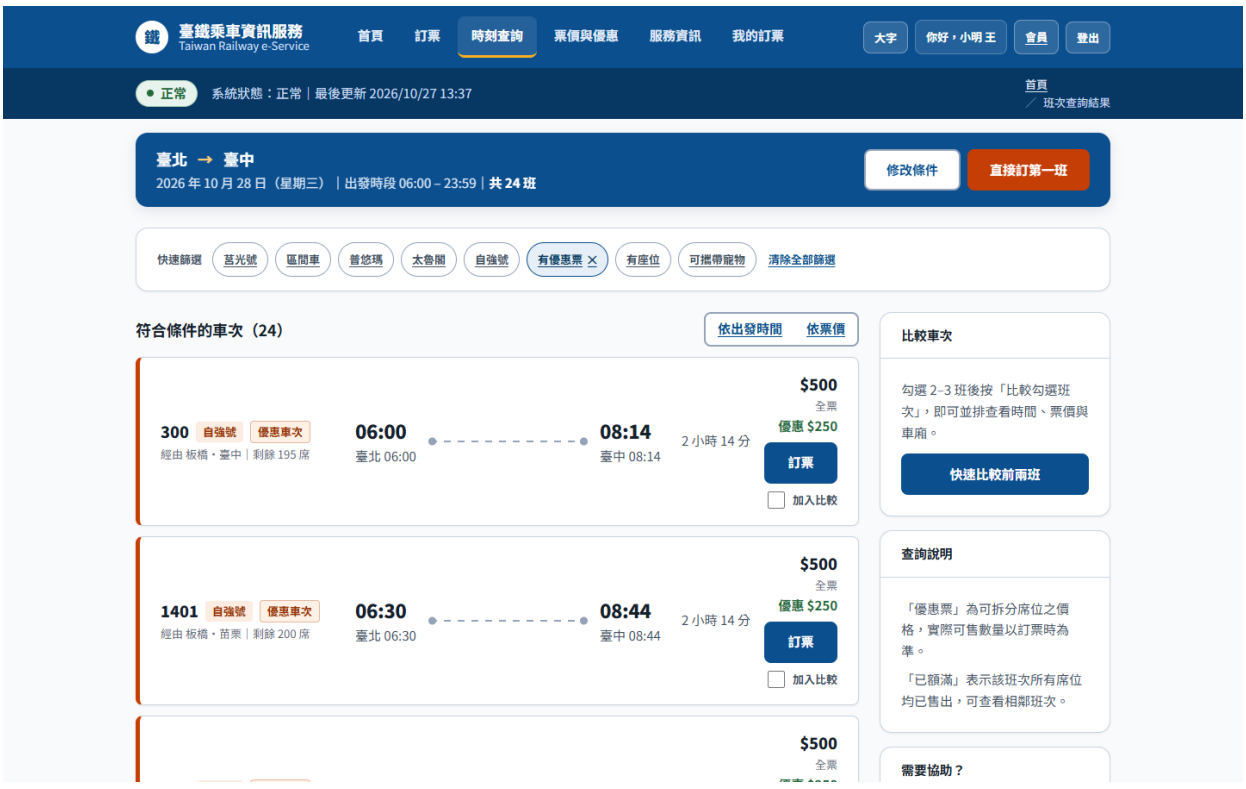


圖 4-3　查詢結果（會員）：查詢摘要列、快速篩選、卡片式班次與右側訂票按鈕；標示全票與優惠票價格。

勾選 2–3 班後可進入比較頁，並排檢視出發 / 抵達時間、行駛時間、經由、票價、車廂種類與電子票適用情形。比較頁的「訂這班」按鈕直接帶入訂票流程，減少來回切換。



圖 4-4　班次比較：自強號 300 與 1401 的出發時間、抵達時間、票價與車廂種類並排呈現。

4.4　會員登入前後差異

登入頁明確說明「登入後才能完成訂票、查看電子票與訂票紀錄」，並在下方提供課程 Demo 測試帳號。登入成功後，導覽列增加「我的訂票」，右上角顯示「你好，小明王」、會員與登出；首頁的提示卡則改成「我的訂票 / 會員資料」與訂票筆數。



圖 4-5　會員登入頁：帳號 / 密碼、註冊連結與測試帳號說明。

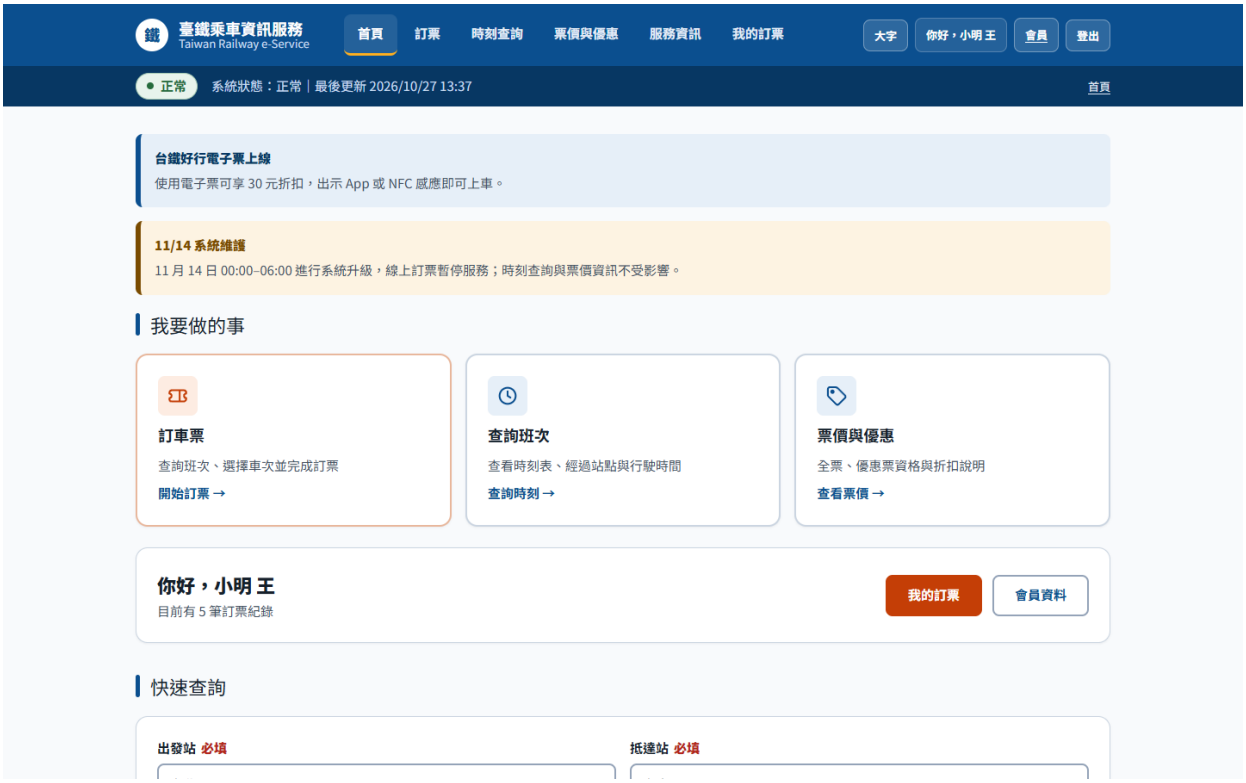


圖 4-6　首頁（會員）：導覽新增「我的訂票」，右上角顯示會員名稱，首頁顯示目前訂票筆數。

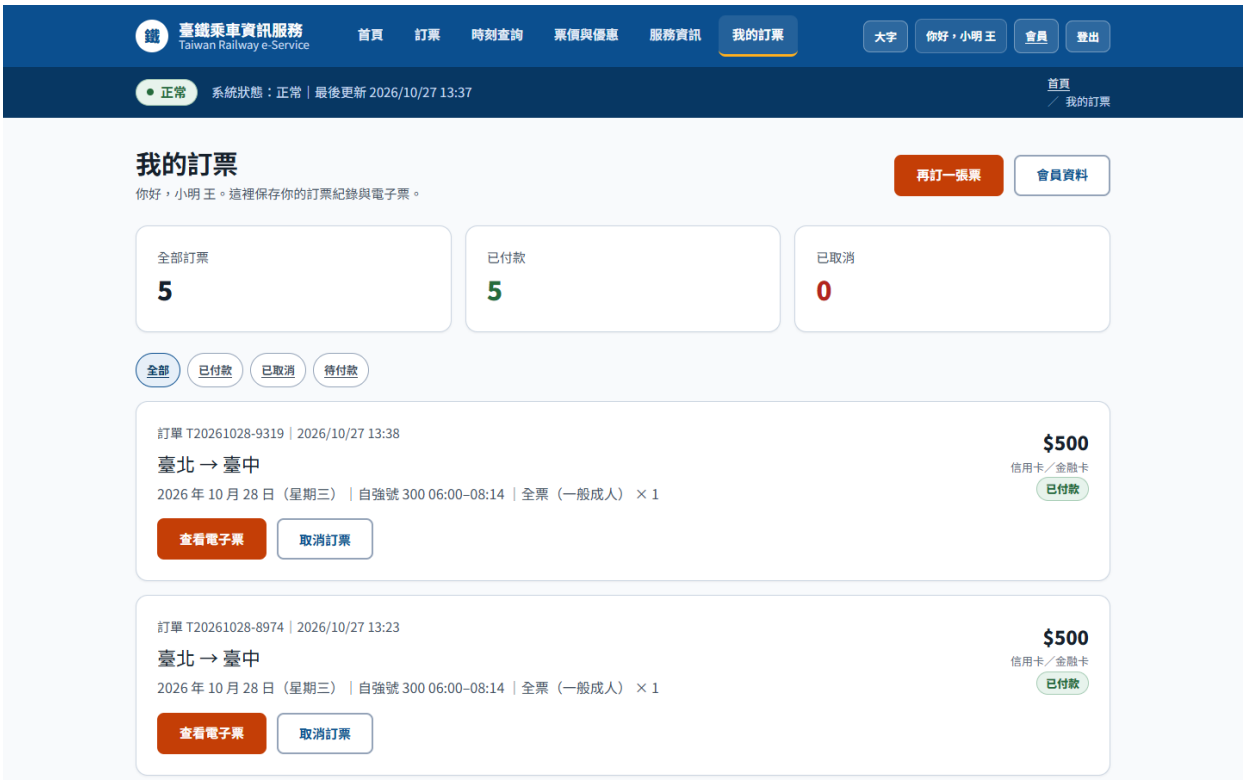


圖 4-7　我的訂票：列出訂單編號、路線、日期、車次、票種、金額、狀態與取消訂票入口。

會員中心另外提供個人資料與常用乘車人。常用乘車人會在下次訂票時自動帶入乘車人表單，減少年長旅客或常客重複輸入證件資料的負擔。

4.5　訂票流程：車廂、票種、乘車人、完成

訂票流程以三步驟指示器呈現。第一步選車廂，已售完的商務車廂仍顯示但不可選，並提供替代建議；第二步選票種與人數，右側即時顯示應付金額與優惠資格；第三步填乘車人與付款方式，欄位下方有格式範例與防詐提示。



圖 4-8　訂票 Step 1：車廂選擇。自由座、指定座、Coleman 與商務車廂並列，顯示票價與剩餘座位。



圖 4-9　訂票 Step 1：票種與方案。全票、優惠票、敬老、學生、兒童、愛心票的價格與資格同時可見。



圖 4-10　訂票 Step 2：乘車人資料與付款。欄位含格式範例、必填標記與「本頁為課程 Demo」的防詐說明。



圖 4-11　訂票完成：成功回饋、訂單編號、訂單明細、取票方式與電子票 QR code。

完成頁提供明確的下一步：「出示 QR code」或「至車站取紙本票」，並保留「再訂一張」與「回首頁」兩個出口，不讓使用者卡在死路。

4.6　票價、服務狀態與大字模式

票價頁以卡片說明全票、優惠票、敬老、學生、兒童與愛心票的資格與應備證件；服務狀態頁在維護時顯示預計恢復時間、影響範圍與替代通路，而不是只有「回首頁」。大字模式會放大字級與間距，但保留相同版面與操作位置。

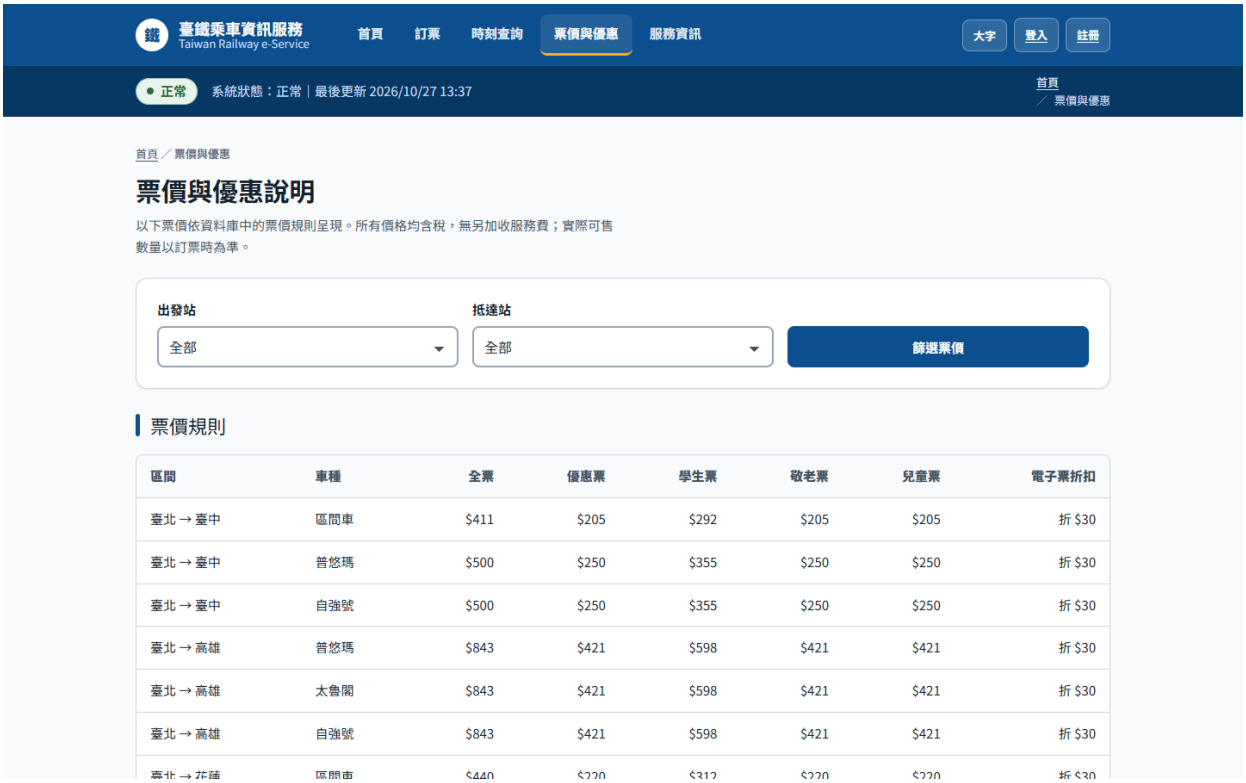


圖 4-12　票價與優惠：票種資格、折扣與應備證件以卡片呈現。



圖 4-13　服務狀態（維護預覽）：顯示暫停服務原因、預計恢復時間與不受影響的服務。

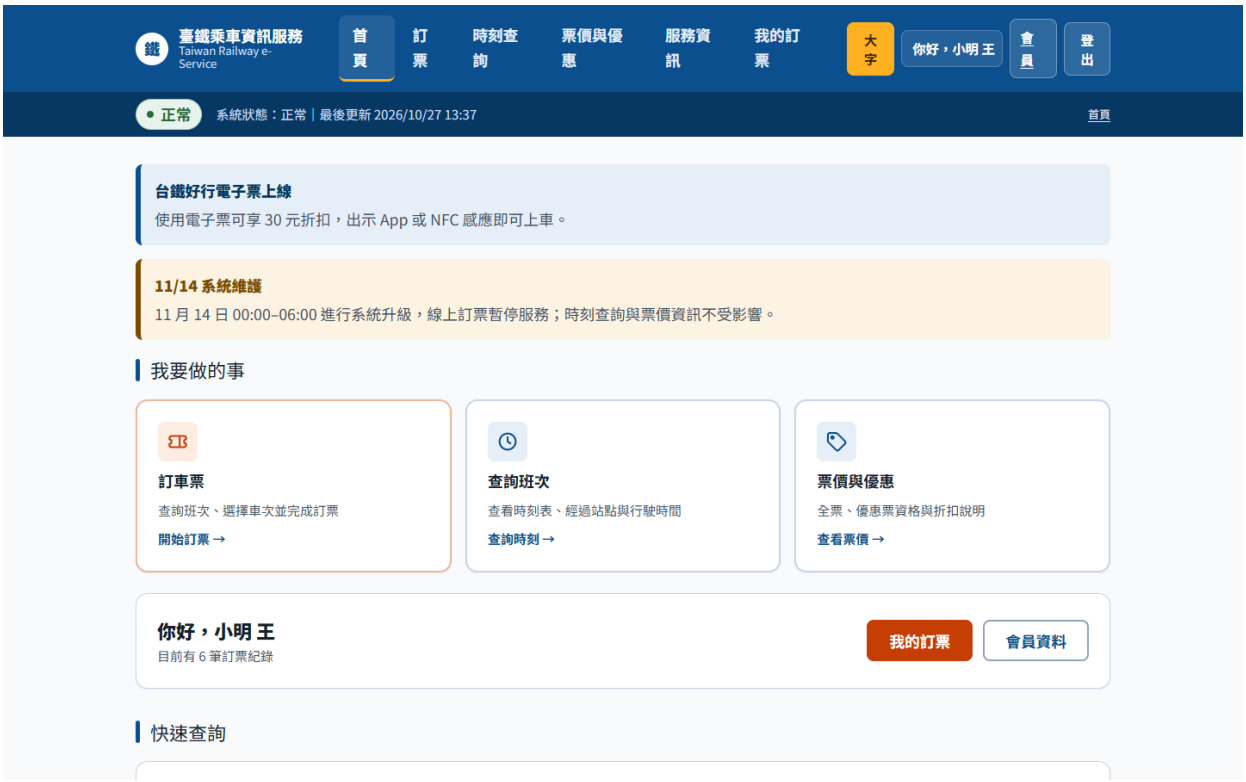


圖 4-14　首頁大字模式：字級與間距放大，主要任務卡與快速查詢仍維持相同位置。

4.7　手機版與無障礙

手機版將導覽改為單列可水平捲動，避免每個標籤被壓成垂直文字；表單欄位改為單欄排列，卡片與按鈕維持足夠點擊面積。這項修正已部署到正式站，並以 iPhone 15 尺寸實際截圖驗證。

正式站原本在 375px 寬度下，主導覽的 5–6 個標籤會被 flex 壓縮，每個中文字被迫斷行，看起來像一排垂直文字。修正方式是在手機斷點把 .nav 設為 flex-wrap: nowrap，並讓每個鏈結 flex: 0 0 auto、white-space: nowrap，超出寬度時改為水平捲動。這樣標籤維持單行，使用者也能滑動看到「服務資訊」與「我的訂票」。



圖 4-15　手機版首頁（iPhone 15）：導覽維持單列可水平捲動，任務卡改為單欄。

查詢表單在手機版改為單欄：出發站、抵達站、日期、時段與票種依序排列，欄位高度維持 48px 以上，避免誤觸。車站欄位仍保留 autocomplete 與「輸入後請從建議清單中選擇」的提示，長輩或首次使用者不必猜測格式。



圖 4-16　手機版查詢表單：車站、日期與時段改為單欄，欄位與按鈕維持可點擊大小。

無障礙設計包含 skip link、main 地標、label for 關聯、必填標記、aria-describedby、focus 外框與 SVG 圖示 aria-hidden。第 7 章會以 axe-core 實際檢測，並誠實列出仍待修正的項目。

5. 後台與資料庫

5.1 Django Admin 管理介面

後台使用 Django Admin，並依網站需求自訂 site header、list_display、list_filter、search_fields、autocomplete_fields 與 actions。管理員登入後可看到「會員、班次、票價與訂單管理」的模型清單，涵蓋前台所有資料來源。



圖 5-1　Django Admin 首頁：13 個模型依「RAILWAY」與「認證與授權」分組，管理員可新增或修改。

表 5-1　Django Admin 管理範圍

模型	主要管理功能	自訂重點
會員 Member	帳號、密碼、權限、會員資料	UserAdmin 擴充手機、生日、身分證、字級偏好、會員等級
車站 / 車種 / 列車	新增、修改、啟用停用、搜尋	autocomplete、list_filter、search_fields
班次 Trip	日期、路線、票價、座位、狀態	TripCarriage inline、date_hierarchy、route 顯示
票種 / 票價規則	折扣、資格、電子票折扣	list_editable、autocomplete
訂單 Booking	訂單、乘車人、付款、狀態	PassengerInline、mark_paid、cancel_selected（取消並退座）
公告 / 服務狀態	維護公告、系統狀態、恢復時間	list_filter、is_active
常用乘車人	會員常用乘客資料	search_fields、autocomplete

5.2　訂單管理與取消退座

訂單列表顯示訂單編號、會員、班次、票種、乘客人數、總金額、狀態與付款方式，並可依狀態、付款方式、票種與建立時間篩選。管理者可勾選多筆訂單執行「標記為已付款」或「取消訂單並釋出座位」。

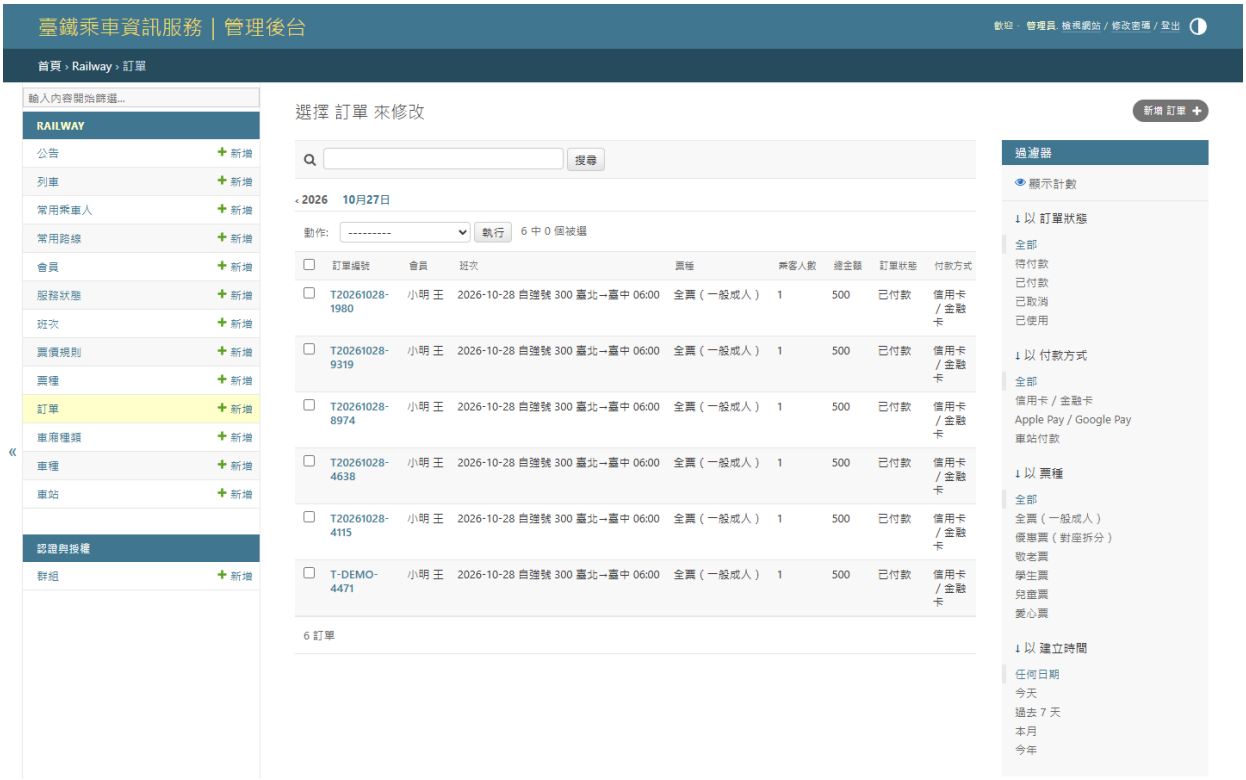


圖 5-2　訂單管理列表：6 筆訂單（截圖時）含新建立的 T20261028-1980，右側提供狀態、付款方式、票種與日期篩選。



圖 5-3　訂單修改頁：可調整會員、班次、車廂、票種、人數、金額、付款與狀態，並檢視乘車人 inline。

取消訂單不是直接把狀態改成「已取消」而已。services.cancel_booking() 會以 select_for_update() 鎖定訂單與班次，檢查是否已取消、是否已過乘車日，再把座位加回 TripCarriage，避免重複退座或超賣。這個設計讓「後台操作」與「前台交易」共用同一套商業邏輯，不會因為從哪個入口取消而產生不同結果。

> **交易安全設計**
>
> 建立訂單：transaction.atomic() + select_for_update() 鎖定 Trip 與 TripCarriage，檢查剩餘座位後才建立 Booking / Passenger 並扣減座位。
>
> 取消訂單：鎖定訂單與班次，檢查狀態與日期，將座位加回，並記錄 cancelled_at。
>
> 若兩個使用者同時訂最後一個座位，後到者會因鎖定與剩餘座位檢查而失敗，系統回傳可理解的錯誤訊息。

5.3 Neon PostgreSQL 架構與連線

資料庫使用 Neon 的 production branch，資料庫名稱 neondb。Django 以 psycopg2 連線，正式環境使用 pooled connection；本機則從 .env.local 讀取 DATABASE_URL。專案層 OpenCode MCP 以 OAuth 連到 Neon，方便在開發時直接查詢 schema 與資料。

表 5-2 Neon 與資料庫組態

項目	內容
Neon 專案	wild-shape-74199559（production branch）
資料庫	neondb
PostgreSQL 版本	PostgreSQL 18.6（aarch64-unknown-linux-gnu）
資料庫角色	neondb_owner
public tables	24 張（含 Django 內建 auth / admin / session 與 railway 自訂資料表）
驅動	psycopg2-binary 2.9.13（本機 psycopg3 對 Neon TLS 交握會失敗，已改用 psycopg2）
連線方式	Render 使用 pooled connection；DB_CONN_MAX_AGE=0，避免免費方案連線被回收後出現 stale connection
環境變數	DATABASE_URL、SECRET_KEY、ALLOWED_HOSTS、CSRF_TRUSTED_ORIGINS、DEBUG=0

5.4 資料表與資料量

下表為 2026/10/27 直接查詢 Neon 的實際資料量。班次與車廂資料由 seed_demo 依執行當日動態產生 15 天內的班次，因此 Trip 與 TripCarriage 的數量較大；訂單則包含課程 Demo 與 smoke test 建立的資料。

表 5-3　Neon 資料量（2026/10/27）

資料表	筆數	說明
railway_member	2	admin、user 兩個測試帳號
railway_station	16	臺北、臺中、高雄等主要車站
railway_train	9	自強號、莒光號、區間車等車次
railway_trip	960	15 天內多條路線的班次
railway_tripcarriage	3,840	每班次四種車廂
railway_tickettype	6	全票、優惠票、敬老、學生、兒童、愛心
railway_fareRule	18	區間 × 車種票價規則
railway_booking	7	含 6 筆 smoke test 與 1 筆手動操作
railway_passenger	7	每筆訂單一位乘車人
railway_savedpassenger	1	會員常用乘車人
railway_announcement	2	電子票上線、系統維護
railway_servicestatus	3	網站、取票、線上訂票

5.5　資料管理範例

以下為 Neon 中實際存在的訂單資料，可用來驗證前台「我的訂票」與後台訂單列表顯示的是同一筆資料。

表 5-4　訂單資料範例（節錄）

訂單編號	會員	乘車日	車次	路線	狀態 / 金額
T20261028-1980	user	2026-10-28	300	臺北 → 台中	paid / $500
T20261028-7459	user	2026-10-28	300	臺北 → 台中	paid / $500
T20261028-9319	user	2026-10-28	300	臺北 → 台中	paid / $500
T-DEMO-4471	user	2026-10-28	300	臺北 → 台中	paid / $500

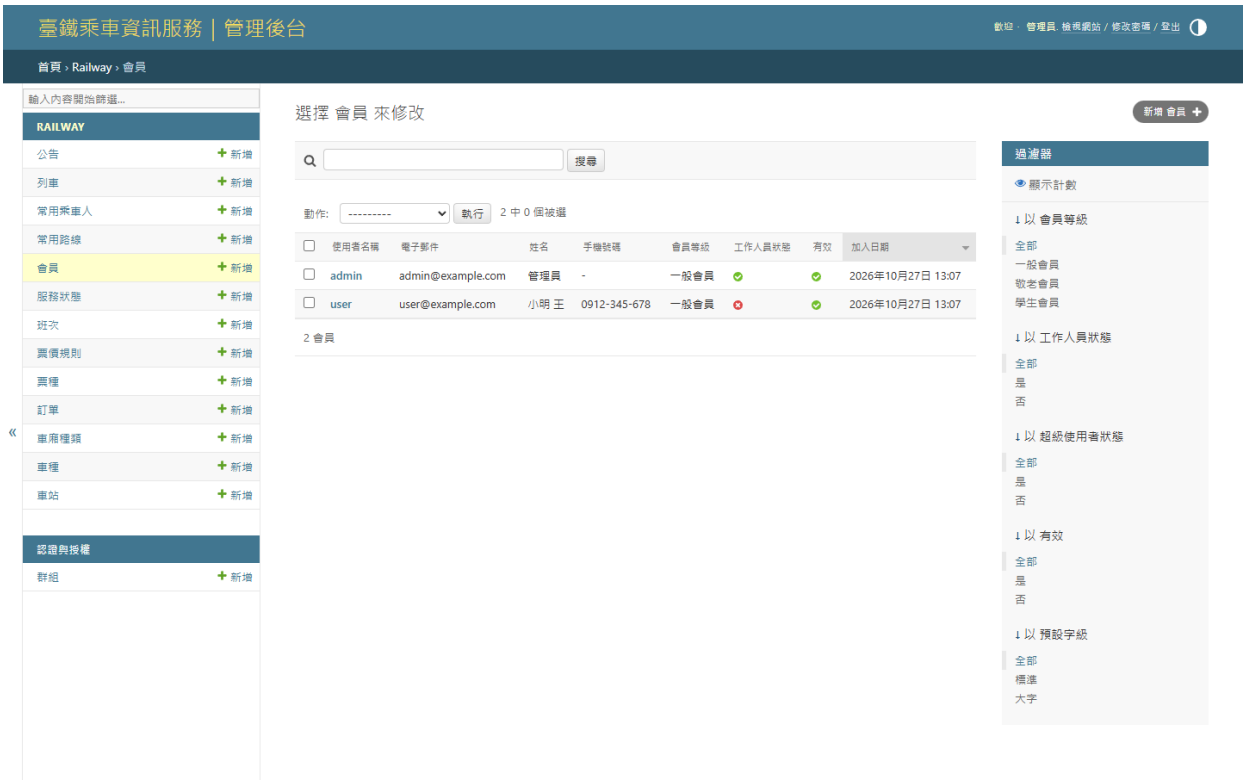


圖 5-4　會員管理：顯示 admin 與 user 的帳號、Email、姓名、手機、會員等級、工作人員狀態與加入日期。

5.6　安全性與資料保護

- **CSRF 與 Session**：所有 POST 表單都帶 csrfmiddlewaretoken；會員登入使用 Django session，登出為 POST 並驗證 CSRF。
- **權限控管**：訂票、我的訂票、會員資料以 login_required 保護；Django Admin 僅 is_staff / is_superuser 可進入。
- **個資最小化**：付款為模擬流程，不儲存信用卡號；身分證與生日僅用於訂票驗證與乘車人資料。
- **秘密管理**：DATABASE_URL、SECRET_KEY 由環境變數注入；.env.local、.neon 已加入 .gitignore，不進版本控制。
- **交易一致性**：訂票與取消退座皆使用資料庫交易與列鎖定，避免超賣與重複退座。

5.7　已知限制

本系統是課程 Demo，以下限制已在報告中誠實列出，並列為後續改善方向：付款未串接真實金流；電子票 QR code 不會連動真實閘門；座位配置以整班 / 整車廂數量為主，未做到單一座位選位圖；Email 使用 console backend，沒有實際寄送；頭像、簡訊通知與多語系尚未實作。

6. AI Prompt 使用紀錄（11 組）

本章收錄 11 組在 hw1 專案中實際使用的 AI Prompt。每一組都包含：對話截圖（節錄）、Prompt 原文、使用目的、AI 回覆摘要、對應實作成果與證據位置。

11 組 Prompt 分別來自五個 OpenCode session：後端開發（ses_e847…）、手機版 RWD 修正（ses_eabd…）、靜態原型雲端部署（ses_eabe…）、Bad UI 分析（ses_ee0a…）與報告排版（ses_ee08…）。卡片上的時間、模型與 Session ID 均為實際值。

> **關於對話截圖的呈現方式**
>
> 為統一版面，本章卡片以相同的 OpenCode 對話樣式重新排版；文字逐字節錄自實際 session 匯出檔，沒有事後改寫或虛構。
>
> 每張卡片都標示 Session ID，可用 `opencode session export <session-id>` 重新匯出完整逐字稿比對。
>
> AI 產出的程式碼與文字均經組員人工覆核，並以實際網站操作、測試或 Lighthouse / axe 檢測驗證後才納入報告。

6.1 後端需求與架構規劃

Session：ses_e847594a3ffdr1ClFJASUDwXda | 時間：2026/10/27 13:27 | 模型：deepseek-v4.1-flash-free



圖 6-1　OpenCode 對話紀錄（需求規劃）：後端需求與架構規劃。

Prompt 原文（節錄）

> *幫我根據期中專題的報告要求完成後端系統，并且可以根據目前的 redesign ui 來設計。先規劃好步奏再開工。然後 neon 你可以幫我安裝 mcp 到這個專案上。*

項目	內容
使用目的	確認專題範圍、技術選型與施工順序，避免直接動工造成架構返工。
AI 回覆摘要	提出 Django + Neon + Render 的三層架構、15 個資料模型、10+ 條路由與 6 階段施工順序；先確認現有 repo 只有靜態原型，再決定以 server-rendered templates 延續 redesign UI。
對應實作成果	docs/BACKEND.md、README 後端章節、railway/models.py、6 階段施工計畫。
證據位置	`docs/BACKEND.md`、`railway/models.py`、`README.md`

此組 Prompt 的關鍵價值，在於先確認「現有 repo 只有靜態原型」，再選擇 server-rendered Django，而不是把 SPA 硬改成 API。後續的 models、views、templates、Admin 與部署都依這個架構展開；如果沒有先做這個判斷就直接寫程式，很容易把 prototype 的 JS 路由與 Django 路由混在一起，造成維護困難。

6.2　測試帳號、Neon 與 Render MCP

Session：ses_e847594a3ffdr1ClFJASUDwXda｜時間：2026/10/27 13:42｜模型：deepseek-v4.1-flash-free



圖 6-2　OpenCode 對話紀錄（環境建置）：測試帳號、Neon 與 Render MCP。

Prompt 原文（節錄）

> *會員帳號：user / user123；管理員帳號：admin / admin123。render 也有 agent，用 opencode 的這個網址 https://render.com/agents/opencode，你可以安裝這個 mcp 到這個專案上。*

項目	內容
使用目的	建立可驗收的測試帳號與雲端工具鏈，讓後續開發、部署與資料庫操作可重現。
AI 回覆摘要	完成 Neon CLI 登入與專案連結（init 外掛安裝失敗時改用 --no-agent-setup）；Neon MCP 寫入專案層 opencode.json；Render CLI / MCP 以 OAuth 設定；建立 Django 專案與虛擬環境。
對應實作成果	seed_demo 測試帳號、opencode.json、render.yaml、neon.ts、.env.local。
證據位置	`railway/management/commands/seed_demo.py`、`opencode.json`、`render.yaml`

6.3　Render 部署完成與 Smoke Test

Session：ses_e847594a3ffdr1ClFJASUDwXda｜時間：2026/10/27 14:04｜模型：deepseek-v4.1-flash-free



圖 6-3　OpenCode 對話紀錄（部署驗收）：Render 部署完成與 Smoke Test。

Prompt 原文（節錄）

> *繼續（接續完成 Render 部署與公開站驗證）。*

項目	內容
使用目的	確認前後台與資料庫真的可用，而不是只在本機通過。
AI 回覆摘要	回報最新部署 dep-dbg3gs2jnfac73f5jt2g 已 live（commit ee10e2c）；公開前台、/admin/、/healthz/ 均可存取；公開站 smoke test 20 項全數 PASS；npm run build 與 npm test 通過，Django 13 個測試 OK。
對應實作成果	Render 前台 / 後台、healthz、scripts/smoke_test.py、13 個 Django tests。
證據位置	第 7 章表 7-2、表 7-3；Render 部署紀錄

6.4　Render MCP 與自動部署診斷

Session：ses_e847594a3ffdr1ClFJASUDwXda｜時間：2026/10/27 14:20｜模型：deepseek-v4.1-flash-free



圖 6-4　OpenCode 對話紀錄（問題診斷）：Render MCP 與自動部署診斷。

Prompt 原文（節錄）

> *MCP 已授權（Neon, Render），render 中我看到有鏈接 github。這樣就完成了嗎？*

項目	內容
使用目的	釐清「看起來已連結」與「實際會自動部署」的差異，避免報告與現況不一致。
AI 回覆摘要	Render MCP 仍回 unauthorized；改用 REST API 確認服務設定為 autoDeploy: yes，但推送 ead1b97、95e759e 兩個空 commit 都沒有建立新部署。原因是 MCP 設定的 {env:RENDER_API_KEY} 沒有被載入，等於送空 Bearer；改為 OAuth，並把 deploy hook 寫入待辦。
對應實作成果	docs/TODO.md、opencode.json OAuth 設定、自動部署問題紀錄。
證據位置	`docs/TODO.md`、`opencode.json`、`git log`

6.5　Render 免費方案冷啟動限制

Session：ses_e847594a3ffdr1ClFJASUDwXda｜時間：2026/10/27 14:26｜模型：deepseek-v4.1-flash-free



圖 6-5　OpenCode 對話紀錄（雲端限制）：Render 免費方案冷啟動限制。

Prompt 原文（節錄）

> *rander 說會有這個限制：Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.*

項目	內容
使用目的	把平台限制轉成報告中的已知限制與展示說明，並找出自動部署的真正問題。
AI 回覆摘要	免費方案閒置約 15 分鐘會休眠，下一個請求需等 30–60 秒；醒來後功能與 Neon 資料正常。建議報告註明首次開啟請稍候，或用 uptime 監控每 10 分鐘 ping /healthz/。同時提醒自動部署尚未生效。
對應實作成果	報告冷啟動說明、healthz 健康檢查、docs/TODO.md 低優先項目。
證據位置	第 7 章表 7-4、`docs/TODO.md`

6.6　手機版「出發時段」UI 修正

Session：ses_eabd1bb2cffeNnF1luy7p59XDG｜時間：2026/10/19 21:21｜模型：deepseek-v4.1-flash-free



圖 6-6　OpenCode 對話紀錄（RWD 修正）：手機版「出發時段」UI 修正。

Prompt 原文（節錄）

> *在手機頁面中，出發時段的 UI 怪怪的。*

項目	內容
使用目的	修正手機版查詢表單的實際操作問題，確保 375px 寬度仍可選取時段。
AI 回覆摘要	在 375px 下出發時段欄位只有 144px，兩個 select 各被壓縮到 56px；原因是 grid-template-columns 寫在 inline style，不會被 media query 覆蓋。改為 class、將時段容器改為 .tz-selects，手機版垂直排列。
對應實作成果	prototype/screens.js、prototype/styles.css、commit fb025af。
證據位置	`commit fb025af`、`assets/redesign/02-A2-search-form.png`

6.7　其他頁面手機版跑版修正

Session：ses_eabd1bb2cffeNnF1luy7p59XDG｜時間：2026/10/19 21:31｜模型：deepseek-v4.1-flash-free



圖 6-7　OpenCode 對話紀錄（RWD 修正）：其他頁面手機版跑版修正。

Prompt 原文（節錄）

> *其他頁面有些也有跑掉（手機版），幫我修正。*

項目	內容
使用目的	一次修完所有已知手機版跑版頁面，避免只修單頁造成不一致。
AI 回覆摘要	將 A3 結果列表的 inline grid 改為 grid--result、首頁快速查詢改為 grid--2 / grid--3；新增 .plan 手機版 flex-wrap，避免車廂與票種卡片擠壓。共修改 2 個檔案、30 行新增、6 行刪除。
對應實作成果	commit fb025af、RWD 斷點、手機版操作截圖。
證據位置	`commit fb025af`、`assets/impl/24-mobile-home.png`

6.8　Redesign 原型雲端部署

Session：ses_eabe3dd01ffdMazP4xSL4YPvri｜時間：2026/10/19 17:41｜模型：deepseek-v4.1-flash-free



圖 6-8　OpenCode 對話紀錄（靜態部署）：Redesign 原型雲端部署。

Prompt 原文（節錄）

> *你做的改良版的網頁我需要部署到雲端讓每個人可以檢視。幫我做的完善一點並告訴我可以用什麼方式部署。*

項目	內容
使用目的	讓組員、老師與同學不用安裝環境就能檢視 redesigned 原型。
AI 回覆摘要	補上 RWD、skip link、focus 管理、prefers-reduced-motion、SEO / OG、manifest；建立 deploy/ 扁平化資料夾、404.html、robots.txt、_redirects、vercel.json；提供 GitHub Pages、Vercel、Netlify、Cloudflare Pages 等部署方式。Lighthouse Accessibility / Best Practices / SEO 皆 1.0。
對應實作成果	deploy/、GitHub Pages workflow、五種部署說明。
證據位置	`.github/workflows/deploy-pages.yml`、`deploy/`、`README.md`

6.9　GitHub Pages 啟用失敗排除

Session：ses_eabe3dd01ffdMazP4xSL4YPvri｜時間：2026/10/19 18:06｜模型：deepseek-v4.1-flash-free



圖 6-9　OpenCode 對話紀錄（部署排錯）：GitHub Pages 啟用失敗排除。

Prompt 原文（節錄）

> *部署時顯示這個 error：Error: Get Pages site failed. Please verify that the repository has Pages enabled and configured to build using GitHub Actions, or consider exploring the enablement parameter for this action.*

項目	內容
使用目的	排除雲端部署的第一個阻礙，讓靜態原型真的能被公開檢視。
AI 回覆摘要	判斷是 GitHub Pages 尚未啟用或未設定為 GitHub Actions；更新 workflow 加上 enablement: true，並提醒 repo 必須是 Public、第一次啟用需 1–2 分鐘。若仍失敗，改到 Settings → Pages 手動選擇 GitHub Actions。
對應實作成果	.github/workflows/deploy-pages.yml、GitHub Pages 公開網址。
證據位置	`.github/workflows/deploy-pages.yml`

6.10　臺鐵官網 Bad UI 擷取與分析

Session：ses_ee0a9d441ffdqsJSBzMgtyuyoj｜時間：2026/10/09 19:05｜模型：deepseek-v4.1-flash-free



圖 6-10　OpenCode 對話紀錄（Bad UI 分析）：臺鐵官網 Bad UI 擷取與分析。

Prompt 原文（節錄）

> *I. Bad UI Case 選擇：選擇 1 個真實網站，提供網站名稱 + 網站連結 + 至少 3 張原始介面截圖..... II. 問題分析：至少分析 5 項問題，且問題需能與後續 User Flow / Redesign 呼應.....*

項目	內容
使用目的	以實際操作與截圖建立可佐證的 Bad UI 問題清單，作為 redesign 的依據。
AI 回覆摘要	實際走訪臺鐵官網並截圖：右側浮動選單文字重疊、Cookie 橫幅遮擋內容；查詢表單控制項在無障礙樹中全是無名稱 generic 節點；直接輸入站名被伺服器拒絕，只回模糊的「格式錯誤」；結果表格的「訂票」欄被浮動選單遮住。
對應實作成果	7 張 bad UI 截圖、P1–P8 問題、Lighthouse 數據、12 張 redesign 分鏡。
證據位置	`assets/badui/`、`assets/redesign/`、`HW1` 報告

6.11　生成式 AI 使用標注

Session：ses_ee087a1e7ffdUvHfxXVACxFs2g｜時間：2026/10/09 02:46｜模型：deepseek-v4.1-flash-free



圖 6-11　OpenCode 對話紀錄（AI 聲明）：生成式 AI 使用標注。

Prompt 原文（節錄）

> *順便在合適的地方加上標注，若使用生成式 AI 協助分析、文字整理或介面設計，須於文件中附上使用工具名稱。*

項目	內容
使用目的	符合作業的 AI 使用揭露規定，同時讓讀者知道哪些內容由 AI 協助、如何覆核。
AI 回覆摘要	加入兩個層級：封面後、目錄前的獨立聲明頁（聲明框 + 工具名稱 / 使用範圍 / 人工覆核方式），以及第 II、III、IV、VII、VIII 節開頭的綠框標注；新增可重用 aiNote() 元件，工具資料集中在 AI_TOOLS / AI_STATEMENT。
對應實作成果	生成式 AI 使用聲明頁、章節 AI 標注、AI_TOOLS / AI_STATEMENT。
證據位置	`build/report.js`、舊版報告第 2 頁

6.12　AI 使用模式與人工覆核

11 組 Prompt 可歸納為四種使用模式：

- **規劃型**：先要求 AI 讀專題要求與現有程式，提出架構、資料模型與施工順序，確認後才動工（第 6.1 組）。

- **產出型**：由 AI 產生 Django models、views、templates、CSS，組員逐項檢視與修改（第 6.2、6.3、6.8 組）。
- **排錯型**：把實際錯誤訊息、Lighthouse 結果或手機截圖貼給 AI，請它診斷原因並提出修正（第 6.4、6.6、6.7、6.9 組）。
- **文件型**：用 AI 整理問題分析、AI 使用聲明與報告版面（第 6.10、6.11 組）。

人工覆核方式包括：回到原始網站實際操作、以 375px 與 1440px 實際截圖、執行 npm run build / npm test / smoke_test、以 axe-core 檢測無障礙、以及逐項對照程式碼與資料庫。AI 產出不是直接貼上，而是經過「重現問題 → 修正 → 再驗證」的循環。

7. 部署與測試結果

7.1　部署架構與環境設定

正式環境部署在 Render 的 Singapore region，方案為 Free。Build command 依序安裝套件、collectstatic、migrate、seed_demo；Start command 以 Gunicorn 啟動 Django，並以 /healthz/ 作為健康檢查。資料庫使用 Neon production branch 的 pooled connection。

表 7-1　Render 部署設定

項目	設定
服務名稱	railway-redesign（Web Service · Python · Singapore · Free）
Build	pip install -r requirements.txt → collectstatic → migrate → seed_demo
Start	gunicorn config.wsgi:application --bind 0.0.0.0:$PORT --workers 2 --threads 4 --timeout 120
Health check	/healthz/（回傳 status 與 database 狀態）
環境變數	PYTHON_VERSION=3.11.9、DEBUG=0、SECRET_KEY（generateValue）、DATABASE_URL、ALLOWED_HOSTS=.onrender.com、CSRF_TRUSTED_ORIGINS=https://*.onrender.com、DB_CONN_MAX_AGE=0
資料庫	Neon PostgreSQL 18.6 / neondb / production branch
版本控制	GitHub main 分支；靜態原型另有 GitHub Pages workflow

7.2　線上入口與健康檢查

以下入口已於 2026/10/27 實際開啟驗證。健康檢查回傳 {"status": "ok", "database": "ok"}，代表 Django 與 Neon 連線均正常。

表 7-2　線上入口驗證

入口	網址	驗證結果
前台首頁	`https://railway-redesign.onrender.com/`	200 · 顯示任務卡、公告與查詢表單
管理後台	`https://railway-redesign.onrender.com/admin/`	200 · admin / admin123 可登入並管理資料
健康檢查	`https://railway-redesign.onrender.com/healthz/`	{"status": "ok", "database": "ok"}
會員登入	`https://railway-redesign.onrender.com/accounts/login/`	200 · user / user123 可登入
查詢結果	`https://railway-redesign.onrender.com/search/results/`	200 · 顯示 24 班並可篩選
專案倉庫	`https://github.com/Yuntech-work/hw1-railway-redesign`	main 分支 · 包含 Django 後端與部署設定

7.3　建置與單元測試

本機執行 npm run build 與 npm test，實際輸出如下（節錄）：

表 7-3　建置與單元測試結果

指令	結果	說明
`npm run build`	PASS	System check identified no issues（0 silenced）；130 個靜態檔案未變更
`npm test`	PASS	Ran 13 tests in 27.171s；OK；使用 Neon test database（--keepdb）

13 個測試涵蓋查詢條件、票價計算、訂票建立、超賣防護、取消退座、會員登入、後台權限與健康檢查等核心邏輯。

7.4　端對端 Smoke Test

scripts/smoke_test.py 以真實 HTTP 流程走一次瀏覽器行為：訪客瀏覽公開頁 → 會員登入 → 查詢明日臺北→台中 → 選車廂 → 選票種 → 填乘車人 → 建立訂單 → 讀取 QR code → 管理員登入後台 → 查詢 Neon。對正式站執行結果為 20 項全數 PASS。

表 7-4　公開站 Smoke Test 結果（20 項）

#	測試項目	結果
1–7	訪客 GET 首頁、查詢、結果、票價、服務狀態、登入、註冊	PASS（HTTP 200）
8	會員 user / user123 登入	PASS · 導向我的訂票
9	查詢明日臺北→台中，確認可訂班次與票價	PASS
10–12	車廂頁、票種頁、乘車人頁	PASS
13	建立訂單並进入完成頁	PASS · 訂單 T20261028-7459
14	訂單寫入 Neon	PASS · $500
15	QR code 產生	PASS · 10,820 bytes
16–17	管理員登入、後台訂單列表	PASS
18–19	Neon 資料庫與 PostgreSQL 版本	PASS · neondb / PostgreSQL 18.6
20	會員資料存在	PASS · user

最終輸出：SMOKE TEST OK；訂單 T20261028-7459、金額 $500、資料庫 neondb（neondb_owner）。這證明前台、後台與 Neon 三層真的串通，而不是只有畫面能開。

7.5　無障礙檢測（axe-core）

除 Lighthouse 外，本專題以 axe-core 4.12.1 對正式站首頁與查詢結果頁執行檢測。首頁 40 項通過、1 項違反、1 項待確認；結果頁 41 項通過、3 項違反、1 項待確認。這比只說「有做無障礙」更可驗證，也讓我們找到仍待修正的地方。

表 7-5　axe-core 檢測結果與待改善項目

頁面	通過 / 違反	違反項目	說明與後續處理
首頁	40 / 1	page-has-heading-one（moderate）	首頁缺少 h1；後續把任務卡標題改為 h1，或新增視覺隱藏 h1。
首頁	—	color-contrast（incomplete）	select 背景為漸層，工具無法判定對比；需人工確認或改為實色背景。
查詢結果	41 / 3	aria-allowed-attr（critical）	8 個篩選 chip 是 <a>，卻使用 aria-pressed；應改為 button 或移除不允許的 ARIA。
查詢結果	—	color-contrast（serious）	「已售完」disabled 文字對比 2.27:1，低於 4.5:1；需加深顏色。
查詢結果	—	page-has-heading-one（moderate）	結果頁缺少 h1；可將「符合條件的車次」改為 h1。

為什麼要列出未修正項目？

無障礙不是「有做就好」，而是需要持續檢測與修正。把 axe 找到的 3 類問題寫進報告，代表我們能區分「已改善」與「仍待改善」，也讓後續組員有明確的修正清單。

這些問題不影響主要交易流程，但會影響螢幕閱讀器與低視力使用者；已列為第 8.4 節未來展望的第一優先。

7.6　操作驗證與截圖對照

本報告的操作截圖皆來自正式站，對應如下：首頁圖 4-1、查詢表單圖 4-2、查詢結果圖 4-3、比較圖 4-4、會員登入圖 4-5、我的訂票圖 4-7、車廂圖 4-8、票種圖 4-9、乘車人圖 4-10、完成頁圖 4-11、服務狀態圖 4-13、後台首頁圖 5-1、訂單列表圖 5-2、訂單修改圖 5-3、會員管理圖 5-4、手機版圖 4-15～4-16。

這些截圖不是設計稿，而是實際登入、查詢、訂票、取消與後台操作後擷取；訂單 T20261028-1980 與 T20261028-7459 也可在後台訂單列表與 Neon 查詢中看到同一筆資料。

7.7　效能、冷啟動與已知限制

Render 免費方案閒置約 15 分鐘會休眠，第一次請求需 30–60 秒喚醒；蘇醒後功能與 Neon 資料均正常。展示時建議先開啟 /healthz/ 喚醒，或設定 uptime 監控每 10 分鐘 ping 一次。靜態檔案由 WhiteNoise 提供，未使用 CDN；圖片與 CSS 已壓縮，桌面版首次載入在正常網路下可接受。

其他限制包括：付款為模擬流程、QR code 不連動真實閘門、未做單一座位選位、Email 使用 console backend、行動版導覽採水平捲動而非漢堡選單。這些都不影響課程 Demo 的主要流程，但已列為後續改善。

7.8　測試結論

整體而言，核心交易流程已通過建置、13 個單元測試與 20 項公開站 smoke test；前台、後台與 Neon 三層均可用。無障礙仍有 3 類 axe 違反與 1 類待確認，屬於「可用但未完美」的狀態，已在報告中誠實揭露。

8. 結論與心得

8.1　改善成果總結

本專題從 Bad UI 分析出發，最後交付一個可操作的全端系統。改善不只發生在視覺，而是把原本會阻斷任務的問題逐一移除：

表 8-1　Bad UI 與重新設計系統的對照

面向	原始網站	本專題成果	改善說明
主要入口	輪播廣告佔滿首屏，查詢表單在下方	三張任務卡作為主要入口	使用者 3 秒內可定位「訂車票」
導覽	7 項主選單 + 右側浮動選單	5 項主導覽 + 工具列；手機單列捲動	消除重複導覽與遮擋
結果列表	10 欄表格，訂票欄被遮住	卡片式列表，訂票按鈕固定右側	由「看得見點不到」改為可直接操作
查詢脈絡	捲動後忘記查詢條件	固定查詢摘要列 + 修改條件	降低認知負荷與誤訂
錯誤訊息	只說「格式錯誤」	即時驗證 + 具體修正指引	使用者知道下一步怎麼做
票價與優惠	規則分散在不同頁	票價頁集中說明資格與證件	減少買錯票與補差額
會員	登入前後差異不明	訪客可查詢、會員可訂票與查訂單	先給價值再要求註冊
後台	無課程可見的後台	Django Admin 管理 13 個模型	資料可管理、可追蹤
資料庫	無	Neon PostgreSQL 18，24 張 public tables	前台與後台共用同一資料來源
異常處理	只有「回首頁」	恢復時間 + 替代通路 + 重試	異常時仍能完成任務
無障礙	表單控制項無名稱	label / aria / skip link / main 地標	仍有餘項待修正，但已大幅改善

8.2　遇到的問題與解法

表 8-2　開發過程的主要問題與解法

問題	現象	解法與結果
psycopg3 連 Neon TLS 失敗	本機連線在 TLS 交握時被中斷	改用 psycopg2-binary 2.9.13，成功連上 PostgreSQL 18.6
Render 自動部署未觸發	push 後沒有建立新部署，只有手動部署	確認 autoDeploy 設定與 GitHub App 權限；把 GitHub Actions + Deploy Hook 寫入 docs/TODO.md
Render MCP unauthorized	MCP 送出空 Bearer	opencode.json 改用 OAuth，不再依賴未載入的 RENDER_API_KEY
手機版出發時段跑版	375px 下兩個 select 被壓到 56px	inline grid 改 class、時段容器垂直排列，commit fb025af
正式站手機導覽斷行	每個導覽標籤被壓成垂直文字	手機版 .nav 改為單列、鏈結 flex:0 0 auto、white-space:nowrap，重新部署驗證
Word / PowerShell 中文編碼	產出的 docx / 終端機輸出出現亂碼	統一 UTF-8、設定 PYTHONUTF8=1、以 Node docx 產生器輸出
Render 免費方案休眠	首次開啟需等 30–60 秒	報告註明冷啟動、以 /healthz/ 喚醒、評估 uptime 監控

8.3　組員心得

曾宥綸｜分析與介面設計

這次最大的收穫是「不要急著畫畫面」。以前看到 Bad UI 會直覺說「配色很醜」，實際走訪臺鐵官網、截圖、看無障礙樹之後，才發現真正的問題是訂票按鈕被遮住、錯誤訊息無法指引修正。把問題編號 P1–P8、再對應到 N1–N8 與實作頁面，讓設計決策有依據，也比較能和組員溝通。

吳嘉恩｜流程與會員功能

User Flow 讓我體會到「異常分支」才是流程設計的重點。原本只畫了順利完成訂票的路徑，被提醒要補「查無班次、車廂售完、證件格式錯誤」之後，才發現這些才是使用者真正卡住的地方。會員登入前後的差異也讓我學會：不是把所有功能鎖起來，而是先讓訪客得到價值，再在需要個資時才要求登入。

陳柏宏｜後端與資料庫

Django 的 ORM 與 Admin 讓後端開發快很多，但最花時間的是資料模型與交易安全。訂票要用 transaction.atomic() 與 select_for_update() 鎖定班次與車廂，取消時要把座位加回且不能重複退座；這些邏輯如果只寫在 view 裡，很容易從不同入口產生不同結果，所以我們把它集中到 services.py。Neon 的連線也踩到 psycopg3 TLS 的坑，改用 psycopg2 才成功。

王慈文｜部署、測試與報告

部署不是把程式丟上雲端就好。Render 的自動部署沒有真的觸發、MCP 的 API key 沒有被載入、免費方案會休眠，這些都不在程式碼裡，卻會直接影響展示。這次學會用 Render API 查部署狀態、用 smoke test 證明前後台與 Neon 真的串通，也學會把「還沒修好」的無障礙項目誠實寫進報告，而不是假裝滿分。

8.4 未來展望

- **無障礙修正**：把篩選 chip 改為 button 或移除 aria-pressed、補上 h1、加深 disabled 文字對比，並重新跑 axe 到 0 violation。
- **自動部署**：完成 GitHub Actions + Render Deploy Hook，讓 push 後自動上線，並在部署後自動跑 smoke test。
- **交易功能**：串接真實金流（需申請商家帳號）、單一座位選位圖、Email / 簡訊通知與電子票加入錢包。
- **使用者研究**：以 5–8 位真實使用者進行任務測試，記錄完成時間、錯誤率與 SUS 分數，驗證 redesign 的實際效果。
- **維運**：加上 uptime 監控、錯誤追蹤（Sentry）與資料備份還原演練。

8.5 結語

這個專題讓我們從「批評一個網站不好用」，走到「做一個真的可以用的網站」。Bad UI 分析提供問題意識，Persona 與 User Flow 提供設計依據，Django 與 Neon 提供可操作的身體，Render 讓它真正被看見。專案還有不完美的地方，但每一個已知問題都被記錄下來，也都有下一步。對我們來說，這比一份只有漂亮畫面的報告更接近真實的介面設計工作。

— 報告完畢 —

附錄 A　路由、帳號與環境變數

表 A-1　完整路由與測試帳號

項目	內容
前台	/、/search/、/search/results/、/compare/、/fares/、/status/
訂票	/booking/<trip_id>/carriage/、/ticket/、/passenger/、/booking/complete/<order_no>/
會員	/accounts/login/、/signup/、/logout/、/my/bookings/、/my/profile/
API / 健康檢查	/api/stations/?q=、/booking/<order_no>/qr.svg、/healthz/
後台	/admin/
會員帳號	user / user123
管理員帳號	admin / admin123
必要環境變數	DATABASE_URL、SECRET_KEY、ALLOWED_HOSTS、CSRF_TRUSTED_ORIGINS、DEBUG=0、DB_CONN_MAX_AGE=0

附錄 B　AI 工具使用與人工覆核

表 B-1　生成式 AI 工具、使用範圍與覆核方式

工具	使用範圍	人工覆核方式
OpenCode（deepseek-v4.1-flash）	專題架構規劃、Django 後端與 templates、CSS / RWD 修正、Django Admin、seed_demo、Render / Neon 設定、smoke test、Word / PDF 報告產生器與排版、AI Prompt 紀錄整理	逐項回到網站實際操作；執行 npm run build / npm test / smoke_test；以 375px / 1440px 截圖；以 axe-core 檢測；對照程式碼與資料庫
Claude（Anthropic）	HW1 的 Bad UI 問題分類、Persona / User Story 草擬、User Flow 異常分支檢核、Bad UI vs. Redesign 對照分析	回到臺鐵官網實測、以 Lighthouse 數據佐證、逐張檢視 12 張分鏡
Neon MCP	查詢 Neon 專案、branch、資料表 schema 與資料量	以 psql / Django ORM 交叉查詢確認
Render MCP / API	查詢服務狀態、部署狀態、環境設定	以公開網址、healthz 與 smoke test 驗證

本報告第 6 章的 11 組 Prompt 均取自本專案 OpenCode session 匯出檔，卡片以統一視覺重排，文字逐字節錄，沒有事後虛構。所有 AI 產出均經組員人工覆核後才納入系統與報告。

附錄 C　參考資料

- 國營臺灣鐵路股份有限公司官方網站：https://www.railway.gov.tw/
- Django 5.2 Documentation：https://docs.djangoproject.com/en/5.2/

- Neon Documentation：https://neon.com/docs/
- Render Documentation：https://render.com/docs
- WCAG 2.1：https://www.w3.org/TR/WCAG21/
- Lighthouse：https://developer.chrome.com/docs/lighthouse/
- axe-core：https://github.com/dequelabs/axe-core
- OpenCode Documentation：https://opencode.ai/docs/

本報告由第 13 組共同完成｜2026 年 10 月 27 日